

SaTC: CORE: Small: HOLMES: Holistic and Hardware-assisted Control-Flow Security for Microcontroller Systems

1 Overview and Objectives

Microcontroller units (MCUs) are small computers designed for embedded and Internet of Things (IoT) applications, in contrast to microprocessors used in personal computers and servers. MCUs power a variety of systems and subsystems, such as sensors, medical devices, industrial computing units, and electronic control units. They are essential to everyday life, and are predicted to reach the 1 trillion count by 2035 [1]. Popular MCU architectures include ARM Cortex-M [2], Tensilica Xtensa [3], Toshiba TLCS [4], Texas Instruments MSP [5], Renesas RX/RL [6] and Infineon AURIX [7]. In particular, the Arm Cortex-M family is the most popular MCU architecture on the market as more than 80 hardware vendors have licensed Cortex-M cores [8]. 4.4 billion Cortex-M MCUs were shipped in the 4th quarter of 2020 alone [9], and it is estimated that Cortex-M MCUs account for almost 100 billion deployed embedded and IoT devices in 2021 [10]. While the benefits of these systems are unparalleled, unfortunately, they are susceptible to cyberattacks that are occurring at unprecedented levels, and which often have severe consequences ranging from loss of life [11] to homeland security breaches [12]. To ensure our embedded and IoT infrastructure is built on a trustworthy and secure foundation, the “IoT Cybersecurity Improvement Act of 2019” (S.734 in the Senate) [13] highlights the need for urgent actions on MCU systems security.

Among the attacks targeting MCU systems, control-flow hijacking remains the most dangerous as it can result in arbitrary code execution. To provide motivation for the proposed research, we conducted an analysis of over 500 bugs of software systems designed for MCUs from 2017 to 2023 [15]. Our findings indicate that more than 52% of these bugs, e.g., out-of-bounds (OOB) access or use-after-free (UAF) vulnerabilities, may lead to control-flow hijacking. Consequently, as illustrated in Figure 1, the disclosed MCU system bugs exhibit significantly higher severity, as indicated by common vulnerability scoring system (CVSS) scores, compared with general bugs [14] reported during the same time periods. This observation underscores the critical importance of addressing control-flow hijacking attacks on MCU systems.

Numerous approaches have been put forth to address control-flow hijacking at multiple levels, including the hardware layer, e.g., data execution prevention (DEP) [16] and Intel’s control-flow enforcement technology (CET) [17], the programming language layer, e.g., the memory-safe Rust language [18], the compiler layer, e.g., stack canaries [19], control-flow integrity (CFI) [20], software fault isolation (SFI) [21], and the system layer, e.g., address space layout randomization (ASLR) [22]. While many concepts and principles behind these approaches are applicable across different architectures, note that most were initially developed for microprocessor systems (e.g., x86/64 and Cortex-A). Consequently, these approaches heavily rely on microprocessor features. For instance, ASLR assumes the presence of a memory management unit (MMU) to provide sufficient entropy, and the absence of an MMU on MCUs significantly diminishes the effectiveness of ASLR [23]. Our premise is that securing the control-flow of MCU systems poses a unique challenge, and porting existing solutions from microprocessor systems to MCU systems becomes infeasible due to hardware and performance constraints. Specifically, we have pinpointed the following characteristics unique to MCU systems, and the corresponding challenges for safeguarding their control flows:

MCU systems have unique security, memory, and performance needs. Existing control-flow security solutions for microprocessor systems have several limitations when deployed on MCU systems: (L1) the

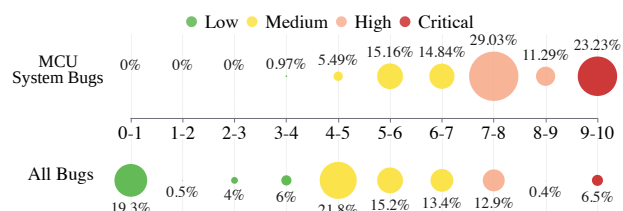


Figure 1: CVSS score distributions (10 representing the most severe). The MCU-related software bugs demonstrate significant higher severity compared to all CVE-assigned bugs [14] between 2017 to 2023.

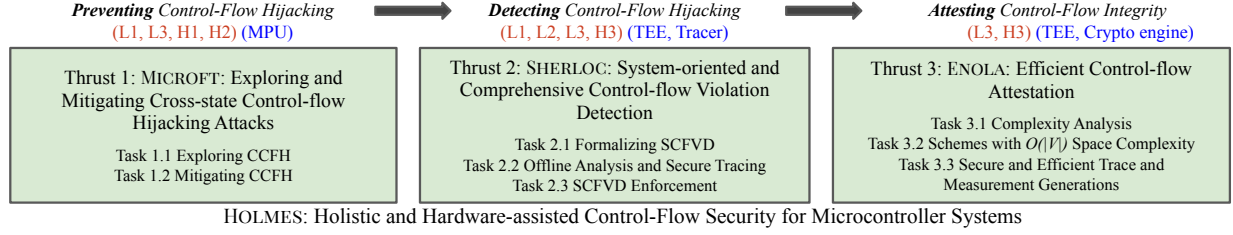


Figure 2: The proposed three-step research thrusts. The to-be-addressed issues and implementation layers are shown in orange and blue, respectively.

software layer on MCU systems typically consists of real-time operating systems (RTOS) or bare-metal systems, where applications run at privileged levels, or directly on hardware without an OS. However, many existing control-flow protection schemes are primarily designed to work for unprivileged applications. Therefore, they are not directly applicable to MCU systems where it is necessary to monitor the control-flow transfers *within* and *among* privileged and unprivileged components. For instance, existing control-flow violation detection (CFVD) approaches used in desktop systems [24–29] require kernel modifications to store and analyze the trace; hence, they can only monitor unprivileged applications; (L2) prior control-flow integrity (CFI) solutions based on inlined instrumentation increase binary size and alter memory layouts, which is often infeasible with small memories of a few hundred Kilobytes. To address this concern, some solutions, such as CaRE [30], opt not to insert, but rather replace function calls and indirect branches with dispatch instructions, resulting in a significant performance overhead; (L3) inlined instrumentation at compile-time typically necessitates access to the source code of the protected system, a requirement that embedded and IoT vendors are often reluctant to fulfill; (L4) control-flow attestation (CFA) attests the execution path of a program. However, existing approaches only work for very small programs or code snippets due to the slow software calculation of measurements, and the substantial size of trace or measurement data, which can grow exponentially with the number of control-flow edges.

MCUs offer limited security and performance guarantees at the hardware layer: (H1) MCUs do not have some of the hardware units we take for granted on microprocessors. For example, they do not have an MMU, without which applications share the same physical address space, making it difficult to enforce isolation, or to implement control-flow hijacking mitigation, such as ASLR; (H2) to achieve efficient execution, MCUs incorporate unique hardware optimizations that give rise to new security concerns. For instance, MCUs’ streamlined trusted execution environments (TEE) usually feature fast state switch mechanisms. In this project, we aim to explore how such streamlined mechanisms can be exploited for control-flow hijacking and arbitrary code execution with escalated privilege in the rich execution environment (REE); (H3) most MCUs operate as single-core processors, while existing security solutions, including the aforementioned CFVD methods [24–29], can only achieve acceptable performance on multicore systems.

Additionally, *the necessity of direct hardware access on MCU systems restricts the effectiveness of memory-safe languages*. MCU systems are typically implemented in low-level languages, such as C, which lack safety features, making them susceptible to memory corruption bugs that can lead to control-flow hijacking [31]. However, transitioning to memory-safe languages is not straightforward, as MCU systems necessitate direct hardware access, such as through memory-mapped registers. For instance, using `unsafe` blocks in Rust is required for such tasks, making them still vulnerable to control-flow hijacking.

1.1 Proposed Research

As shown in Figure 2, this proposal describes a comprehensive three-step research agenda, namely HOLMES: Holistic and Hardware-assisted Control-Flow Security for Microcontroller Systems, which has the following overarching scientific goal driving the proposed research: *Designing and developing novel and efficient algorithms and systems that prevent and detect control-flow violations for an entire MCU*

software system and provide control-flow attestation for a substantial portion of the system. To achieve *holistic* prevention and detection, HOLMES aims to safeguard not only the forward and backward edges of both unprivileged and privileged programs, but also the control-flow transfers among unprivileged and privileged components, including software components in the TEE. In pursuit of attestation for a significant portion of the system, HOLMES aims to utilize hardware measurements, and offer trace and measurement sizes with linear space complexity relative to the number of basic blocks. We plan to realize this vision by leveraging novel MCU-specific hardware functionalities (hence, *hardware-assisted*), such as streamlined TEE, memory protection unit (MPU), simplified tracer, and lightweight crypto engine, through three main thrusts:

Thrust 1: MICROFT: Exploring and Mitigating Cross-state Control-flow Hijacking Attacks. MCU TEEs support fast state switches through function calls and returns, resulting in an unlimited number of entries between secure and non-secure privilege levels. While faster, the security implications of this state switch mechanism have not been thoroughly studied. In this thrust, we will first explore how this mechanism can be exploited for control-flow hijacking with escalated privilege in the non-secure state. To mitigate the attacks, we will design an MPU-assisted address checking approach and a more efficient software-fault isolation-based address masking approach. The novel mitigation approaches tackle the root of the problem by bridging the semantic gap between the TEE and the REE.

Thrust 2: SHERLOC: System-oriented and Comprehensive Control-Flow Violation Detection. CFI is a fundamental security property. The key challenge is designing secure and efficient forward- and backward-edge CFI enforcement algorithms and systems for bare-metal and RTOSs for MCUs. Research efforts thus far have focused on optimizing certain aspects, e.g., only backward-edge, and have not yet been directed toward comprehensive and deployable solutions for MCU devices. We propose to build a new solution based on trace-based violation detection, which previously was limited to unprivileged program monitoring. The limitations of existing trace-based approaches will be addressed with the combination of TEE-governed secure tracing and novel interrupt- and scheduling-aware detection algorithms. The novel algorithms and approach anchor the security and efficiency in the strong protection of trace records, the offloading of tracing to debugging units, and the reduced number of context switches.

Thrust 3: ENOLA: Efficient Control-Flow Attestation. Control-flow attestation can attest the execution path of a program. However, existing approaches face a significant challenge in the slow software-based measurement calculation, and in transmitting a substantial amount of measurement or trace data, which can either be exponential to the number of basic blocks or linear to the runtime trace. Consequently, they can only handle small programs or code snippets. In this thrust, we propose to analyze the interrelationship of complexity at each step of CFA, and establish the theoretical bounds governing these relationships. Additionally, we propose an efficient CFA algorithm and solution that operates efficiently on MCUs and achieves linear space complexity for measurement and trace data concerning the number of basic blocks, ensuring scalability for larger MCU programs. In addition to contributions in algorithm design, the novel solution will leverage the lightweight crypto engines for hardware-assisted measurement.

Intellectual Merit. Our proposed project will represent a significant advancement in algorithm, mechanism, and system design for safeguarding the control-flow of MCU systems. Specifically, the project initiates by delving into the realm of cross-state control-flow hijacking, seeking to both comprehend and counteract this phenomenon to secure control-flow from TEE to rich execution environment. Following that, the project leverages the capabilities of the secured TEE and hardware tracing units to develop comprehensive control-flow violation detection mechanisms on the entire MCU software. Finally, the project capitalizes on the secured TEE and crypto engines to devise a space-efficient algorithm to attest the control-flow integrity of a significant portion of the MCU software.

1.2 Relevance to SaTC

This project is highly-relevant for SaTC as it will have significant impact on the security of MCU systems, which are pervasive in smart home, transportation, healthcare, public safety, and many other sectors. Holistic, effective, time-efficient, and space-efficient control-flow security algorithms, mechanisms, and systems (keyword: *Software* and *Intrusion Detection*) using new MCU-specific hardware functionalities (keyword: *Systems*) will provide much more effective security, and can be deployed more readily to protect a wide array of MCU devices.

1.3 PI Qualifications

Our team consists of two experienced and well-established researchers who have made significant contributions to the fields of (embedded) systems and software security [32–48]. **PI Zhao**, an Associate Professor at the Khoury College of Computer Sciences at Northeastern University, has focused extensively on embedded system security. His research group has been actively involved in systematically studying the security issues on MCUs. For instance, they analyzed the hardware and architectural limitations of MCU systems and their ability to support the implementation of modern security solutions [49]. Additionally, they crawled thousands of real-world MCU firmware samples to identify and analyze the security vulnerabilities present in them [50]. **Co-PI Kirda**, a Professor at the Khoury College of Computer Sciences at Northeastern University, is a recognized expert in software and systems security, with substantial experience in both areas. His group introduced SHiFT [47], a semi-hosted fuzz testing framework for embedded applications, and developed D-Box [48], a DMA-enabled compartmentalization solution for embedded applications. The team’s research has garnered several awards, including the **SACMAT Test-of-Time Award 2024**, and **distinguished paper awards** at **USENIX Security 2019**, **NDSS 2011**, **AsiaCCS 2022**, and **CODASPY 2014**. In terms of research community service, Lead PI Zhao co-founded the Symposium on Vehicle Security and Privacy (VehicleSec) as modern vehicles are full of MCUs. The Co-PI has served as program chair or co-chair for prominent conferences, including CCS, USENIX Security, and NDSS. Furthermore, Co-PI Kirda has an excellent track record of transferring research results to industry, and has built popular public systems such as ANUBIS[] and EXPOSURE[] in the past. The team also has a track record in open-sourcing prototypes [40, 43, 51–53] and transitioning prototypes to patents and products [40, 54–56], e.g., in Samsung Tizen OS [57].

2 Background: Microcontroller Units

Architecture. The majority of MCU architectures adhere to a reduced instruction set computer (RISC) architecture, and lack a memory management unit (MMU). For instance, a Cortex-M MCU has 16 general-purpose registers, i.e., R0 to R12, R13/SP (stack pointer), R14/LR (link register), and R15/PC (program counter). LR holds the return address for a subroutine or a special value EXC_RETURN when an interrupt occurs. Cortex-M MCUs have thread and handler execution modes. They also have privileged and unprivileged levels. Execution in the thread mode can be either privileged, or unprivileged, whereas the handler mode is always privileged. The current execution mode and privilege level are determined by the combination of the interrupt program status register (IPSR) and the CONTROL register.

Streamlined Trusted Execution Environment. A TEE provides isolated execution, confidentiality, and integrity of applications executing within the environment. MCU TEEs are typically streamlined versions of their microprocessor counterparts, designed for higher efficiency and simplicity. For example, while Cortex-A and Cortex-M TrustZone share similar high-level concepts, the underlying mechanisms in Cortex-M are simplified. Indeed, environment switching, AKA state switching, on Cortex-A is accomplished by the secure monitor call (SMC) instruction and special secure monitor mode, whereas the division of TEE and REE in Cortex-M is memory map based, and transitions between environments take place faster and automatically through secure gateway (SG) and branch and call instructions (BLNS and BLXNS).

Simplified Tracer. MCUs come with instruction and data tracers. For example, Cortex-M MCUs have a micro trace buffer (MTB) unit. The MTB can trace non-sequential program counter changes (e.g., branches

and exceptions) by storing trace packets to the trace buffer located on the memory. When the buffer is full, the MTB can either halt the processor, or overwrite from the beginning of the buffer.

Lightweight Crypto Engine. A crypto engine provides cryptographic services, such as message authentication codes. For instance, the pointer authentication (PA) security extension on Cortex-M [58] can generate and verify a keyed tweakable pointer authentication code (PAC) for a pointer or data. The state and privilege level of execution determines which keys are used. For instance, when PAC/PACG instructions are executed in the unprivileged level and non-secure state, the PAC_KEY_UN_NS register serves as the key for computing the PAC. However, if these instructions are executed in the privileged non-secure state, the PAC_KEY_P_NS register is utilized. These registers are not memory mapped, and can only be modified using the privileged MSR (move-to-system-register) instruction.

3 System and Threat Models

In general, our focus is on addressing control-flow hijacking attacks, but the specific scenarios vary depending on the objectives of each thrust. The scenarios change as the project progresses, with each thrust building on the previous one. In the first thrust, which revolves around exploring and mitigating cross-state control-flow hijacking, the adversaries' aim is to manipulate the services provided by the TEE, potentially escalating their privileges within the non-secure state. We consider memory corruption vulnerabilities in the TEE software due to its development in C and assembly. As a defense measure, we take into account the availability of an MPU. After securing the TEE software in the first thrust, the second thrust focuses on system-oriented control-flow violation detection, where the protected system can be either a bare-metal system or an RTOS running multiple tasks. We anticipate that both privileged and unprivileged modules in REE may contain bugs, and their memory could be corrupted, potentially leading to control-flow hijacking attacks. Similar to other CFI solutions, we assume the code integrity of the protected system, ensuring the tamper resistance of direct calls and branches. Additionally, we assume the presence of a hardware tracing unit. In the third thrust, which centers on control-flow attestation, the attested program operates within the REE, while the measurement engine functions in the TEE. Here, we again place trust in the TEE software. Furthermore, we consider the availability of hardware-based hash functionality.

4 Research Thrusts

The scientific objective of this work is to design and develop novel algorithms, mechanisms, and systems that secure the control-flow of MCU systems.

4.1 Thrust 1: MICROFT: Exploring and Mitigating Cross-state Control-flow Hijacking Attacks

Problem Statement. In contrast to microprocessor TEEs, e.g., Intel SGX or ARM Cortex-A TrustZone, which indicate the state of execution in special system registers, the division of states in MCU TEE is based on memory regions. When running code in the secure memory, the MCU is in the secure state. Otherwise, the MCU is in the non-secure state. As shown in Figure 3, state switches on microprocessor TEEs must, as a result, go through a single entry point – a special execution mode – via a special instruction, e.g., SMC for Cortex-A, whereas state switches in MCU TEE can occur through function calls and returns, resulting in an unlimited number of entries between secure and non-secure privilege levels. While faster, the security implications of this state switch mechanism have not been thoroughly studied, which leads to crucial research questions that we aim to address: *(1) Can the fast state switch mechanism of MCU TEE be exploited for cross-state control-flow hijacking with escalated privilege in the non-secure state? (2) If this is the case, how can we effectively and efficiently prevent such*

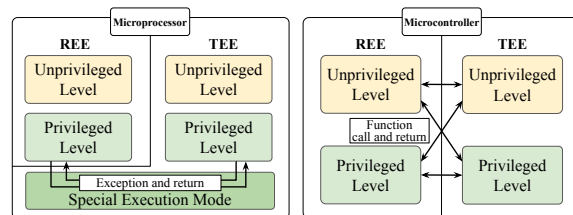


Figure 3: Comparing the state switch mechanisms on microprocessor and microcontroller

attacks? Specifically, we will (i) study the feasibility of cross-state control-flow hijacking attacks on the MCU TEE (Task 1.1); (ii) design and develop MICROFT, which instruments checks before unsafe MCU state switch instructions (Task 1.2).

Related Work by Others. Boomerang [59] is a class of vulnerabilities discovered on microprocess TEEs, i.e., Cortex-A, which allows malicious REE applications to read and write kernel memory. Boomerang, however, does not directly lead to cross-state control-flow hijacking. Return-to-user (ret2usr) attacks redirect compromised kernel pointers to code residing in user space. In response, approaches have been proposed to prevent arbitrary control-flow transfers and dereferences from kernel to user space [60], including Arm Cortex-A’s Privileged Execute Never (PXN). Unfortunately, PXN, or an equivalent feature is not supported by most MCUs. For instance, even for ARM, only the upcoming Cortex-M55 [61] and Cortex-M85 [62] are planned to include PXN support. In contrast to related work, we will be the first to explore and mitigate the cross-state control-flow hijacking attacks on MCUs.

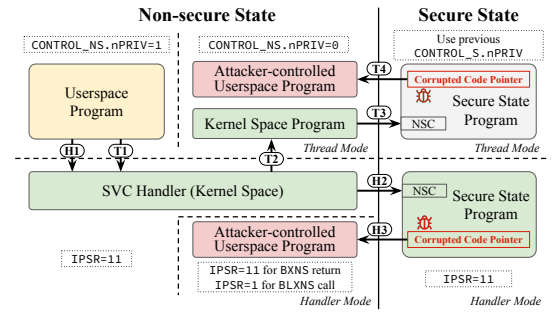
Task 1.1: Exploring Cross-state Control-flow Hijacking for Privilege Escalation (CCFH). Our key insight is that CCFH is likely to be effective in any MCU TEE implementation that permits direct control-flow transfers from a secure state to non-secure userspace programs while continuing to execute at a privileged level.

For instance, on Cortex-M, we observed that the current privilege level is determined by the IPSR register. However, this register is not banked between the secure and non-secure state, which means both states share the same register. As a result, we discovered that the IPSR value of the secure state can be mistakenly carried on to the non-secure state, leading to possible privilege escalation. As shown in Figure 4a, we envision two types of CCFH attacks based on the non-secure execution mode that an attack originates from: handler-mode-originated and thread-mode-originated attacks. Attacks in the former category are more likely to happen in RTOSes, whereas attacks in the latter category are likely to occur in security-enhanced bare-metal systems that support privilege separation. Attacks in either category can be further attributed to an indirect branch case using BXNS, or an indirect call case using BLXNS, resulting in four variants of CCFH attacks.

For example, a possible attack path is handler-mode originated. In this case, a userspace program under the attacker’s control makes a supervisor call (H1), so the processor enters the handler mode, and IPSR is updated to 11 (the interrupt number of SVC). The SVC handler in turn calls an NSC function (H2), and the processor switches to the secure state. Because IPSR is shared between the secure and non-secure state, the secure state program keeps executing in the handler mode with privilege. In a legitimate control path, when the secure state program exits back to the non-secure state using BXNS, the control returns to the SVC handler. However, if the BXNS instruction uses a corrupted code pointer as the destination, the processor can return to any location, e.g., userspace program (H3), in the non-secure state, and keep executing it in the handler mode with privilege.

Building on our findings from Cortex-M, we will explore the design and implementation of MCU TEE systems from Tensilica Xtensa [3], Toshiba TLCS [4], Texas Instruments MSP [5], and Infineon AURIX [7].

Task 1.2: Mitigating Cross-state Control-Flow Hijacking. The key to preventing CCFH attacks is to disallow the execution of non-secure userspace programs at the privileged level, which requires bridging the semantic gap between state and non-secure states. As mentioned earlier, if PXN or an equivalent feature is available, this may be achieved with negligible overhead by properly configuring the MPU regions with PXN. However, there are two limitations of the PXN approach: (1) most MCUs, including the majority of ARM MCUs and those from other vendors, e.g., Tensilica, Toshiba, TI, and Infineon, with significant



(a) Example CCFH attacks on the Cortex-M TrustZone

market share, do not have the PXN or an equivalent feature; (2) for MCUs that support PXN, only a small number of memory protection regions are supported (e.g., 8 or 16), which is not fine-grained enough for complex RTOSs.

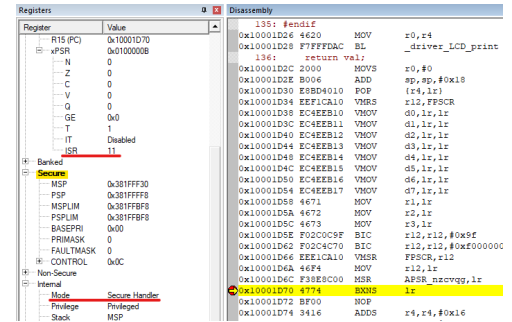
To address these issues, we propose two mechanisms, namely (i) MPU-assisted address checking and (ii) software-fault-isolation based address masking, which can effectively mitigate CCFH attacks on any platform. The proposed approaches stem from our previous work and observation that only the following two control-flow transfers from the secure state to the non-secure state are illegal: (1) any return or call from the secure privileged mode to a non-secure userspace address, regardless of the non-secure state privilege level; (2) any return or call from the secure unprivileged mode to a non-secure userspace address when the non-secure state is at the privileged level. Therefore, we propose to instrument destination address checks at two types of locations: (1) the epilogues of all non-secure callable functions, specifically on ARM Cortex-M architectures, prior to their BXNS instructions; and (2) before calls into non-secure state functions, such as the BLXNS instructions within secure state programs on ARM Cortex-M.

MPU-assisted Address Checking. In this approach, we assume that the non-secure state already adequately implements memory protection mechanisms by configuring userspace and kernel space memory regions with the non-secure MPU. We propose to examine the access permissions of the non-secure destination address by querying the MPU settings. This approach is not intrusive to the non-secure state userspace and kernel space program since they stay intact and do not need to be re-compiled.

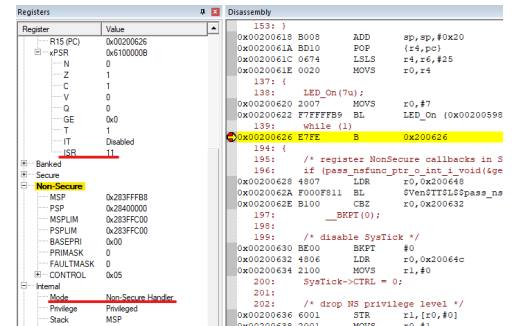
For instance, on ARM Cortex-M, our instrumentation will first check the values of the IPSR register and the nPRIV bit of non-secure CONTROL_NS register. If IPSR is zero, or CONTROL_NS.nPRIV equals one, the control transfer must be legal. Otherwise, the instrumentation will use the test target alternate domain instruction (TTA) to check the MPU attribute of the destination address. From the result, the instrumentation can acquire (1) whether the destination address is within a software-defined MPU region rather than the background region; and (2) the corresponding MPU region number. If the MPU region number is invalid, it means the destination address is either in the default kernel space background region or the EXC_RETURN value from the NSC's return address, both of which are legal. Otherwise, the instrumentation retrieves the MPU attributes on this region and checks the unprivileged execution permission on the destination address.

Software-fault Isolation based Address Masking. The MPU-assisted address checking is generic, but not optimal in terms of efficiency. To address this issue, we also propose an efficient address masking mechanism. We assume that the userspace program and kernel space program are placed in different and known memory regions. The instrumentation simply performs a bit-wise masking on the destination address after checking the non-secure privilege level to force the destination address to fall into the allowed address range.

We anticipate a number of architecture-specific challenges for this approach. For instance, on ARM Cortex-M, a challenge is that a valid destination address of a BXNS instruction could be EXC_RETURN, so the address making mechanism should take care of this case, and does not mask an EXC_RETURN value for BXNS. To handle this case on Cortex-M, we propose to first determine the non-secure privilege level by



(a) The secure state is running in the handler mode before state switching.



(b) After switching to the non-secure state, the MCU keeps running in the handler mode.

Figure 5: Proof of the feasibility of CCFH on Cortex-M

examining `IPSR` and `CONTROL_NS`. If the non-secure state is at the unprivileged level, the control flow is legitimate. Otherwise, the instrumentation further verifies if the return address is `EXC_RETURN` for the `BXNS` case. If the return address is not `EXC_RETURN` or it is for the `BLXNS` case, the instrumentation performs a bit-wise masking operation to ensure the resulting address falls into the designated kernel space.

Building on our initial findings and designs for Cortex-M, we will investigate the design and implementation for other MCU TEE systems.

Preliminary Results on Cortex-M. We have assessed the feasibility of handler-mode-originated CCFH attacks on the AN505 IoT kit image for the ARM MPS2+ FPGA prototyping board with a Cortex-M33 microcontroller [63]. The experiments confirmed that the attacker can escalate privilege, and execute arbitrary code by exploiting the CCFH vulnerabilities. We have responsibly disclosed the preliminary findings of the CCFH attacks on Cortex-M to ARM. Figure 5 is the screenshots of a hardware debugger connecting to the ARM MPS2+ FPGA prototyping board, showing a successful execution of the CCFH attack on Cortex-M. As shown in Figure 5a, prior to the enactment of the `BXNS` instruction, the CPU operates in the secure handler mode, denoted by an `IPSR` value of 11. At this juncture, the address within the `LR` register has been manipulated to reflect the attacker-controlled function in the non-secure state. As shown in Figure 5b, following the state transition the `IPSR` register maintains its value of 11, facilitating the continued execution of the CPU in the non-secure handler mode, now under the control of the attacker’s code.

Other Research Challenges. Building on our previous research experience on fuzz testing for MCUs [47], we aim to explore novel approaches to automatically discover CCFH vulnerabilities in real-world production systems across a wide range of MCU platforms.

4.2 Thrust 2: SHERLOC: System-oriented and Comprehensive Control-Flow Violation Detection

Problem Statement. Control-flow integrity is a basic security property that prevent control-flow hijacking by dictating that indirect control transfers, e.g., indirect call/branch and return, must follow a predetermined control-flow graph (CFG). The property can be enforced by inlined instrumentation, or trace-based control-flow violation detection (CFVD). Existing approaches in both categories for MCU systems are either incomplete (e.g., only consider unprivileged code or backward-edges), or insecure (e.g., trace can be overwritten by adversaries). This raises an important research question: *How can we design a secure and comprehensive control-flow integrity enforcement algorithm and mechanism on forward- and backward-edges of both privileged and unprivileged code for MCU systems?* Specifically, we will (i) formalize the problem of system-oriented CFVD (Task 2.1); (ii) design and develop SHERLOC, which is the first system-oriented CFVD solution (Task 2.2 and Task 2.3).

Related Work. *Inlined instrumentation* inserts a label at a destination and a check before a source [20]. It also inserts shadow stack [20, 64, 65] or return address [66] checks. It is, however, impractical for MCU systems because: (i) instrumentation increases the binary size and changes the memory layout, which is infeasible for MCU devices with limited memory. CaRE [30] preserves the memory layout with a 513% performance overhead; (ii) shadow stacks need to be protected. RECFISH [67], which only protects unprivileged code, maintains shadow stacks at the privileged level and introduces a 30% overhead. Silhouette [68] and Kage [69] use unprivileged store instructions to achieve a low overhead of 3.4% and 5.2%, respectively; nonetheless, Silhouette only works for bare-metal applications, and Kage only supports a small number of tasks. *Trace-based control-flow violation detection* does not instrument the target software. Existing approaches, e.g., CFIMon [25], FlowGuard [27], GRIFFIN [28], PT-CFI [29], nevertheless have these limitations: (i) they only monitor unprivileged code because traces need to be stored at privileged memory, which leads to three issues: (1) they do not apply to MCU systems, which mostly execute at the privileged level; (2) to reduce the number of context switches, their violation analysis is only triggered by an incomplete list of system calls [25–29]; (ii) they rely on advanced debugging features, e.g., trace filtering of Processor Trace (PT) [70], to filter legitimate asynchronous noises [27–29], e.g., interrupts, and context switches. However, these features are not available on MCU hardware debugging units.

Task 2.1: Formalizing System-oriented Control-Flow Violation Detection. Current CFVD mechanisms only monitor one application. We model the interprocedural CFG of an application $\mathcal{A}$ as $G_{\mathcal{A}} = (V_{\mathcal{A}}, E_{\mathcal{A}})$, where $V_{\mathcal{A}}$ is the set of basic blocks and $E_{\mathcal{A}}$ is the set of control-flow transfers defined by $\mathcal{A}$. Existing approaches ignore synchronous and asynchronous control-flow transfers of other applications or the kernel. We model a trace record as a 2-tuple $r = \langle s, d \rangle$ where s is the source address and d is the destination. An ACFVD solution verifies each indirect control-flow transfer must match an edge in the $G_{\mathcal{A}}$:

Application-oriented CFVD (ACFVD). Given the trace $R_{\mathcal{A}} = (r_0, r_1, \dots, r_n)$ of an application $\mathcal{A}$, ACFVD verifies that $r_i \in E_{\mathcal{A}}, \forall i \in \{0, 1, \dots, n\}$.

Due to the lack of memory virtualization, MCU systems have all software components residing in the same physical address space. In current practice, a software system for an MCU, including an RTOS and applications/tasks, is compiled and statically linked into a single binary. As a result, a bug anywhere may lead to serious consequences. Furthermore, MCU systems are interrupt-driven, with the majority of processing taking place in interrupt service routines (ISR). Therefore, it is critical to provide system-oriented CFVD that can monitor control-flow transfers both within and among privileged and unprivileged components.

However, an SCFVD solution faces many challenges as interrupts and task scheduling occur asynchronously, and cannot be anticipated through static CFG analysis and/or dynamic CFG training [27, 29]. Figure 6 depicts an example with an RTOS kernel and three tasks in a single physical address space. Even if the interprocedural CFG can include an over-approximated indirect control-flow transfer set, asynchronous interrupts can happen anytime (e.g., $\langle c_7, t_1 \rangle$). Additionally, unlike a function or interrupt return, where each return goes to the most recent callsite or interrupted instruction, the scheduler may yield to any running task. For instance, both $\langle s_2, b_5 \rangle$, i.e., scheduling task B, and $\langle s_2, c_7 \rangle$, i.e., scheduling task C, are legitimate. Therefore, an interrupt- and scheduling-aware CFVD mechanism is necessary.

We model an MCU software $\mathcal{S}$ including a kernel $\mathcal{K}$ and tasks $\mathcal{T}$ as $(G_{\mathcal{S}}, I_{\mathcal{K}}, Y_{\mathcal{T}})$, where $G_{\mathcal{S}}$ is its interprocedural CFG, $I_{\mathcal{K}}$ is the set of *asynchronous* kernel interrupt service routine addresses, and $Y_{\mathcal{T}}$ is the set of task entry or re-entry addresses. In particular, $G_{\mathcal{S}}$ is modeled as $(V_{\mathcal{S}}, E_{\mathcal{S}})$, where $V_{\mathcal{S}}$ is the set of basic blocks of tasks and the kernel, and $E_{\mathcal{S}}$ is the set of control-flow transfers defined by the tasks and the kernel. $E_{\mathcal{S}}$ also models *synchronous* exceptions that involve control-flow transfers across privilege levels, such as system calls. At system boot, $Y_{\mathcal{T}} = Y_{\mathcal{T}\mathcal{E}} \cup Y_{\mathcal{T}\mathcal{R}}$ is initially composed of statically retrieved entry addresses of all tasks $Y_{\mathcal{T}\mathcal{E}}$, which may be dynamically replaced by re-entry addresses $Y_{\mathcal{T}\mathcal{R}}$ when context switches occur. The SCFVD solution verifies each indirect control-flow transfer must match an edge in the $G_{\mathcal{S}}$ or the destination of each asynchronous control-flow transfer must match an address in the $I_{\mathcal{K}}$ or $Y_{\mathcal{T}}$:

System-oriented CFVD (SCFVD). Given the trace $R_{\mathcal{S}} = (r_0, r_1, \dots, r_n)$ of a system $\mathcal{S}$ including a kernel $\mathcal{K}$ and tasks $\mathcal{T}$, SCFVD verifies that $r_i \in E_{\mathcal{S}} \vee r_i.d \in I_{\mathcal{K}} \cup Y_{\mathcal{T}}, \forall i \in \{0, 1, \dots, n\}$.

Task 2.2: Offline Analysis and Secure Tracing. To validate an indirect or asynchronous control-flow transfer, SHERLOC will require an over-approximated CFG, a list of asynchronous ISR addresses, and entry and re-entry addresses of all RTOS tasks. An over-approximated CFG may result in higher false negative rate, but it guarantees no false positive. A challenge is that the offline analysis can only produce the set of legitimate indirect forward edges and entry addresses of all RTOS tasks. But, the indirect backward edges, i.e., returns, and the re-entry addresses must be maintained dynamically.

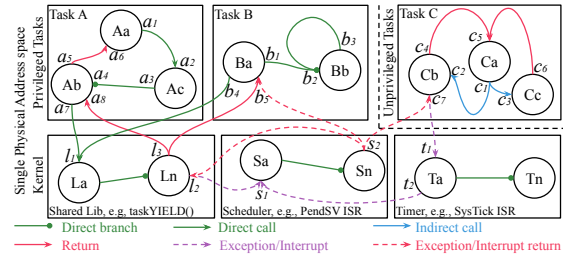


Figure 6: Example system with an RTOS kernel, two privileged tasks, and one unprivileged task

Offline Analysis. To obtain an over-approximated CFG, we propose to leverage existing binary analysis tools, such as angr [71]. Such tools employ forced execution for adding basic blocks, backward slicing to identify contexts, and symbolic back-traversal to resolve indirect calls and branches. Prior research has demonstrated that the outcomes are sufficiently precise in real-world scenarios [26, 72–76]. Afterwards, the offline analysis module reduces the CFG to an indirect branch table (IBT) that contains the source and destination address pairs of all indirect calls and branches. IBT will provide higher precision compared to target set-based approaches [25], as it maintains a destination address set for each source address. As we do not assume advanced task filtering features of the hardware tracing unit, another challenge lies in identifying entry addresses of the running tasks by examining the trace at run-time. To overcome this challenge, we propose to use RTOS heuristics to identify the entry addresses of tasks. For instance, on FreeRTOS, the entry addresses have a `TaskFunction_t` type, and tasks are created using the `xTaskCreate()` function or its variants. These functions call the `prvInitialiseNewTask()` function and pass the task entry address as the first parameter. Based on these heuristics, we will use binary analysis techniques to identify the entry addresses of all tasks, which are kept in a task entry list (Y_{TE}).

Secure Tracing. When an MCU boots, the execution begins with the secure state reset handler as shown in Figure 7a. We propose to partition the secure and non-secure memory, and ensure that the protected system runs in the non-secure state while SHERLOC runs in the secure.

For instance, Figure 7b presents an example memory map of the entire system for Cortex-M. Memory-mapped SAU and MTB registers will be set as secure to prevent non-secure state reconfiguring the memory layout or disabling the tracing. The trace buffer will also be set as secure to restrict access to only SHERLOC. We propose to use the watermark mechanism of the tracer to trigger SHERLOC. A challenge is that even if SHERLOC can immediately suspend tracing, several records generated by housekeeping instructions will be saved in the trace buffer. To prevent these records from overwriting the protected system’s trace, the watermark value should be smaller than the trace buffer size.

Task 2.3: System-oriented CFVD Enforcement. We propose to handle the following types of trace records to make the CFVD interrupt- and scheduling-aware.

Calls/Branches and Function Returns. For indirect branches and calls, SHERLOC will verify whether the source and destination address pair is in the IBT. If not, a CFI violation is detected. Additionally, for both direct and indirect calls, we propose to reconstruct call stacks using a structure named reconstructed call stacks (RCS) for the violation detection of function returns. We propose to use one RCS for bare-metal systems and task- and kernel-specific RCSs for RTOSs. In this way, SHERLOC will provide the same level of precision as shadow stacks for backward edges [30, 68, 69, 77].

Non-context-switch Interrupts, Exceptions, and Their Returns. If a trace record originates from a non-context-switch interrupt, such Non-PendSV interrupts or exception on Cortex-M, we will first compare the destination address d in the record with the addresses in the non-secure state vector table. If no match is found, a CFI violation is detected. If a match is found, we will consider that the interrupt or exception is legitimate. To ensure the return to the interrupted code address from an interrupt or exception, SHERLOC will push the interrupted code address s onto the current task’s RCS. If the interrupt or exception occurs before any task is scheduled, we propose to push the interrupted address onto the kernel RCS. Handling non-context-switch interrupt and exception returns is similar to handling function returns, except that SHERLOC will also peek the top of the current RCS and check whether the destination address matches the top of the RCS. If there is a match, it indicates a legitimate non-context-switch return. Otherwise, for a bare-metal

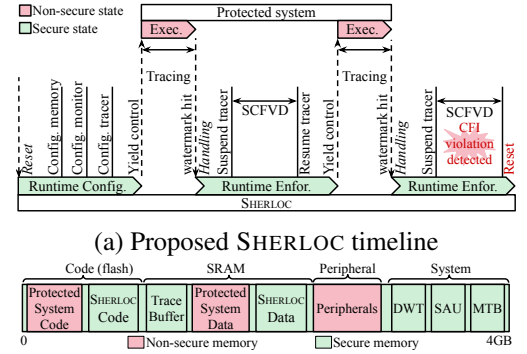


Figure 7: Timeline and memory layout

system where there is only one RCS, it means a CFI violation is detected. For an RTOS, SHERLOC further verifies whether the record is a legitimate context-switch, such as PendSV on Cortex-M, interrupt return.

Context-switch Interrupts and Scheduler Returns. The biggest challenge in SHERLOC will be how to handle context switches, interrupts, such as PendSV on Cortex-M, and scheduler returns. We found that context switches on RTOSs can be triggered by either preemptive scheduling or a task yielding control (i.e., `taskYIELD()` function in FreeRTOS). If the interrupted address is in a shared library, when context-switch ISR returns, it is critical to determine which task- or kernel-specific RCS to use for validation. To facilitate, in addition to the interrupted code address s , we propose to add the current RCS’s top item to the entry and re-entry list $Y_{\mathcal{T}}$. For a context-switch interrupt return record, SHERLOC will check whether the return address is in $Y_{\mathcal{T}}$. If not, a violation is detected. If the address is in $Y_{\mathcal{T}}$, SHERLOC will check whether it belongs to a task or the kernel code and determines which RCS to use next. However, if the address is in the range of a shared library, SHERLOC will need to figure out which task or kernel called that library.

Preliminary Results. Like other CFVD mechanisms, SHERLOC will introduce detection latency, which can be measured by the number of instructions the attacker executes between the hijacked control-flow transfer and its detection. Assuming watermark is N , and most records consume 8 bytes each. If we denote the average number of instructions before a non-sequential control-flow transfer instruction as S , the expected average detection latency can be approximated as $\frac{N \times S}{16}$ instructions. We conducted an empirical measurement of S in FreeRTOS and bare-metal systems. The results show that for ARMClang, FreeRTOS has an S of 5.29 (O3) and 4.57 (Oz). Bare-metal systems have an S of 4.33 (O3) and 3.53 (Oz). We consider a scenario where the attacker attempts to exfiltrate data by conducting a ROP attack, such as returning to (i) `printf()` and (ii) LED toggling. The results show that it takes more than 2,300 instructions to reuse `printf()` to print out just one byte to the UART interface. Alternating LED to leak a byte of data necessitates 440 instructions. Therefore, as estimated in Figure 8, the detection latency of SHERLOC will not be a significant issue.

Other Research Challenges. We will further investigate event-triggered CFVD enforcement, providing a trade-off between security and performance. Similar to the existing implementations of ACFVD [24–28], the enforcement can be triggered by some sensitive events, such as calling a certain kernel API (similar to system calls on desktop) and modifying a particular system register (e.g., changing memory permissions). To this end, we will utilize available debugging units and does not rely on the tracer watermark mechanism.

4.3 Thrust 3: ENOLA: Efficient Control-Flow Attestation

Problem Statement. Control-flow attestation is a technique for precise attestation of the execution path of a program. However, existing approaches, such as C-FLAT [78] and OAT [79], face a significant challenge in slow software-based measurement calculation, and in transmitting a substantial amount of measurement or trace data, which can either be exponential to the number of basic blocks or linear to the runtime trace. Consequently, they can only handle small programs or even code snippets called operations, i.e., a self-contained function, making them unsuitable for MCUs due to limited computational power and network bandwidth. This situation poses a fundamental research question: *How can we develop an efficient control-flow attestation algorithm and system that operates efficiently on MCUs and achieves linear space complexity for measurement and trace data concerning the number of basic blocks, ensuring scalability for larger programs?* Specifically, we will (i) analyze the space and time complexity of existing approaches, examine the interrelationship of complexity at each step of CFA, and establish the theoretical bounds governing these relationships (Task 3.1); (ii) design and develop ENOLA, which will be the first control-flow attestation solution with $O(|V|)$ space complexity for trace and measurement data (Task 3.2 and Task 3.3).

Task 3.1: Complexity Analysis of Control-Flow Attestation. We observed that the complexity of

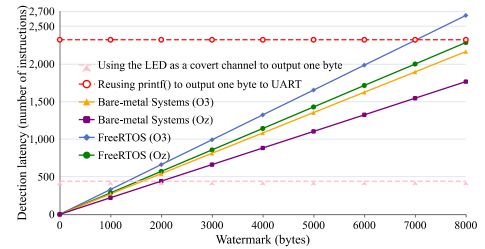


Figure 8: Estimated detection latency

	Complexity Analysis			Evaluation Environment		
	Trace [†]	Measurement [†]	Verification [‡]	CPU	Apps (optimization levels)	Evaluation Size
Naive trace-based	$O(l)$	$O(1)$	$O(l)$	n/a	n/a	n/a
OAT [79]	$O(n)$	$O(1)$	$O(l E)$	Multi-core microprocessors	5 IoT apps including <i>syringe pump</i> [80] (OO)	$n < 1,000$
C-FLAT [78]	n/a	$O(2^{ V })$	$O(2^{ V })$	Multi-core microprocessors	<i>Syringe pump</i> (OO)	$ E = 322$
LO-FAT [81]	n/a	$O(2^{ V })$	$O(2^{ V })$	Customized multi-core microprocessors	<i>Syringe pump</i> (OO)	$ E = 322$
Blast [82]	$O(2^{ V })$	$O(1)$	$O(2^{ V })$	Multi-core microprocessors	Embench [83] and <i>syringe pump</i> (OO)	$ V < 987$
ENOLA (Preliminary)	$O(V)$	$O(1)$	$O(2^{ V })$	Single-core MCUs	Will evaluate on full embedded crypto libraries	

Table 1: Preliminary space[†]/time[‡] complexity analysis. l represents the number of non-sequential control-flow transfers, n denotes the number of forward transfers in a trace. $|V|$ is the total number of basic blocks, and $|E|$ corresponds to the number of edges in a CFG.

existing approaches has not been thoroughly analyzed, impeding the research community’s overall ability to pinpoint the bottleneck. Our initial focus will be on formalizing CFA, and assessing its inherent complexity, enabling us to define the objectives for ENOLA: in control-flow attestations, a remote verifier $\mathbb{V}$ requests a prover $\mathbb{P}$ to execute a program $\mathcal{P}$ by sending a challenge c to ensure freshness. $\mathbb{P}$ executes $\mathcal{P}$, and a trusted measurement engine $\mathbb{E}$ within $\mathbb{P}$ generates an attestation report $\mathcal{R}$, which is transmitted to $\mathbb{V}$. $\mathcal{R} = (Auth, Sig_K(Auth, c))$ is composed of a cumulative authenticator $Auth$ of the control-flow path and a signature over $Auth$ and c using a key K . The authenticator $Auth = (T, M)$ comprises of a full or partial trace T of the control-flow path and a measurement M computed over the control-flow path.

We model the program $\mathcal{P}$ ’s interprocedural CFG as $G_{\mathcal{P}} = (V, E)$, where V represents the set of basic blocks and E signifies the set of control-flow transfers. Each edge $e_i \in E$ has a source address $e_i.s$ and destination $e_i.d$. A full execution trace $T_{\mathcal{P}}$ is a sequence of non-sequential control-flow transfer destinations $(e_1.d, \dots, e_l.d)$. We employ n to represent the count of forward branch instances and m to signify the count of backward branch instances (i.e., returns). Thus, $l = n + m$. Usually, there exist more forward branch instances than backward ones (i.e., $n > m$), and significantly more forward branch instances than basic blocks in $G_{\mathcal{P}}$ (i.e., $n \gg |V|$). The following are our preliminary complexity analysis results of some example approaches.

Full-trace based approach. In a naive trace-based attestation scheme, the authenticator $Auth$ consists of the full trace $T_{\mathcal{P}}$, and the measurement is the cryptographic hash of the full trace. The space complexity of the trace and measurement combined is $O(l)$. Subsequently, $\mathbb{V}$ abstractly executes $\mathcal{P}$. During abstract execution, when a non-sequential control-flow transfer site is encountered, $\mathbb{V}$ verifies whether the next record in $T_{\mathcal{P}}$ belongs to the destination set of the site i , denoted as $\cup e_i.d$. The time complexity for verification is $O(l)$ as well. Due to the theoretically unbounded nature (e.g., considering an infinite loop) and practically substantial size of l , this approach is impractical, even for small programs running on embedded systems.

OAT. To reduce the trace size, OAT [79] employs three techniques: (1) for each conditional branch, OAT utilizes a single bit to denote if the branch is taken or not. Compared to recording the entire address, this technique results in a significant reduction in size (constant factor smaller) although maintaining the same asymptotic space complexity; (2) for each forward indirect branch, i.e., indirect jump and indirect call, OAT records its destination address; (3) for backward edges, OAT maintains a single chained hash value of return addresses. During abstract execution, $\mathbb{V}$ tracks the branch direction using the bit to indicate whether it’s taken or not for conditional branches. It validates destination addresses for indirect branches, and calculates and verifies the hashes for return instructions. OAT reduces the trace and measurement size complexity to $O(n)$, and the verification time complexity for $\mathbb{V}$ in OAT is $O(l|E|)$.

C-FLAT and LO-FAT. In C-FLAT [78] and LO-FAT [81], the $Auth$ only consists of measurements but excludes trace information entirely (i.e., $T = \emptyset$). With this approach, if $\mathcal{P}$ has no loops the measurement has only a single measurement value with a space complexity of $O(1)$. However, the measurement space complexity in C-FLAT and LO-FAT is $O(2^{|V|})$ considering loops. This is due to the fact that the measurement space complexity is bounded by the number of paths in loops, which is itself bounded by $O(2^{|V|})$. Additionally, because there is no trace available, $\mathbb{V}$ needs to explore all possible paths to verify $Auth$, resulting in a time complexity of $O(2^{|V|})$.

Blast. To further reduce the trace size, Blast [82] does not generate traces at the block level; rather, it operates at the function level. To address loops, Blast employs Ball-Larus’ technique of resetting the path number to a non-zero value at the back-edge of the loop. This approach divides the CFG into a series of acyclic components, with each component independently computing path numbers. Therefore, for a CFG with loops, the size of a function’s trace comprises of a set of path numbers, which in the worst case is the number of paths in the function, bounded by $O(2^{|V|})$. The measurement in Blast is a hash of the trace.

Insights from the preliminary analysis. As shown in Table 1, due to either the substantial size of the trace or measurement data needed to be sent by $\mathbb{P}$, i.e., $O(n)$ trace in OAT, $O(2^{|V|})$ measurement in C-FLAT and LO-FAT, and $O(2^{|V|})$ trace in Blast, existing approaches can only handle whole but small programs or even snippets called operations, i.e., a self-contained task or logic. Therefore, the space complexity of authenticators remains a fundamental limitation impacting the scalability of previous approaches. For example, OAT was evaluated on operations that contained fewer than 1,000 forward branches (n), C-FLAT was tested for programs with 322 edges ($|E|$), respectively, and Blast was evaluated for programs with at most 987 basic blocks ($|V|$). Furthermore, LO-FAT requires modifications to the CPU and cannot be implemented on off-the-shelf devices.

Remaining research challenges. CFA is a sequence of operations where each step influences the time and space complexity of subsequent steps. Building on our preliminary results, we will analyze the inter-relationship of complexity at each step and establish the theoretical bounds governing these relationships, thereby providing a foundation to guide future designs.

Task 3.2: Trace and Measurement Schemes with $O(|V|)$ Space Complexity. Another objective and significant challenge of ENOLA is to minimize the size of trace and measurement data. To this end, we propose a novel authenticator: $Auth = (T, M)$ that includes a basic block occurrence trace (T_O) and measurements $M = \langle M_f, M_b \rangle$ that encompass the forward (M_f) and backward path (M_b). A preliminary design of T_O is structured as follows: $(\langle v_1.s, \#v_1 \rangle, \dots, \langle v_{|V|}.s, \#v_{|V|} \rangle, t_1, \dots, t_x)$. Here, $v_i.s$ represents the start address of the basic block v_i , and $\#v_i$ indicates the occurrence count of v_i during an execution. Legitimate indirect branch/call targets correspond to the start addresses of all basic blocks. However, in cases involving non-legitimate targets, such as the middle of an instruction in return-oriented programming (ROP) attacks, T_O includes a list of target addresses in $t_1, \dots, t_x$. Essentially, any appearance of t_i in T_O represents a control-flow violation. In practice, x will be a very small number, and ENOLA can also establish a maximum limit for x . Therefore, the space complexity of the trace is linear with respect to the number of basic blocks, i.e., a complexity of $O(|V|)$. M will be composed of two cumulative measurements for both forward path. Therefore, the size of measurements are constant with a space complexity of $O(1)$.

Task 3.3: Secure and Efficient Trace and Measurement Generations with TEE and Crypto Engine. The next key challenge is to securely and efficiently generate the trace and measurements on MCUs. To this end, ENOLA will comprise of four key components: the compiler, code scanner, measurement engine, and verifier. The ENOLA compiler will generate the binary program under attestation from source code, and incorporate necessary instrumentation. The instrumentation serves two primary functions: Firstly, it is responsible for reporting control-flow transfer events in program $\mathcal{P}$ to the ENOLA measurement engine $\mathbb{E}$, which runs in the secure state. This enables the measurement engine to construct T_O . This will be done by instrumenting instructions calling into the secure state before each non-sequential control-flow transfer. Secondly, the instrumentation will utilize crypto instructions to calculate the measurements M_f and M_b . Another challenge is that we cannot store the measurements in the memory as they could be compromised by memory corruptions. To tackle this issues, the ENOLA compiler will reserve two general-purpose registers to store the measurements. Furthermore, the ENOLA compiler will produce an over-approximated CFG that the ENOLA verifier utilizes for abstract execution. Additionally, we will design a code scanner to ensure that no program running in the REE, including the program under attestation and all other modules (e.g., libraries and kernel), has system instructions that could be used to tamper with the crypto engine key registers.

Other Research Challenges. As shown in Table 1, the preliminary design of the backtracking veri-

fication algorithm is expected to exhibit a time complexity of $O(2^{|V|})$, akin to C-FLAT. Considering that the verifier will be running on powerful desktops or servers, this approach will be feasible for a significant portion of an MCU system, although it may not cover the entire MCU system. Exploring optimization techniques for this complexity presents another avenue for research.

5 Evaluation Plan

Evaluation and Synergistic Collaborations. The research outcomes will be evaluated in terms of security, effectiveness, performance, and cost. With our collaborators, we will evaluate and integrate the proposed algorithms and systems into real-world MCUs and the products of Samsung. **A collaboration letter is attached:** Dr. YongHo Hwang, Corporate Vice President and Head of Security, Samsung Electronics.

Implementation Plan. We will implement the proposed algorithms, mechanisms and systems for ARM Cortex-M, Tensilica Xtensa, Toshiba TLCS, Texas Instruments MSP, Renesas RX/RL, and Infineon AURIX architectures. The implementation will build on our experience in Cortex-M4 [43, 84, 85], Xtensa [47], and Renesas RX/RL [47]. The proposed program analysis and instrumentation algorithms will be implemented as LLVM passes. We will develop the proposed binary analysis and vulnerability detection algorithms on the top of Ghidra [86] and angr [87].

Collaborative Evaluation and Deployment. We will partner with Samsung to evaluate the security, effectiveness, performance, and cost on benchmark applications and real-world products. *Security* and *effectiveness* will be assessed by formal and informal analysis and by checking the effectiveness of mitigating known attacks on MCU applications and OSs, including Samsung Tizen OS [57], FreeRTOS [88], and Mbed OS [89]. *Performance* will be quantified through theoretical analysis and run-time overhead measurement of the algorithms and benchmarks, including BenchIoT [90], BEEBS [91], and compared with prior works. *Cost* will be measured by the memory and storage overhead on MCU devices.

6 Broader Impacts

MCU CTF Platform and Challenges. Existing CTF platforms, including pwn.college [92], OverTheWire [93], lack the architectural support and challenges for MCU systems. They are designed for the Intel x86/64 architecture only. The PIs are developing and will host PWNIoT.ACADEMY, a next generation and open CTF platform that supports challenges for MCU architectures. In particular, the PIs will develop control-flow hijacking and attack mitigation challenges for MCU systems, e.g., buffer overflow, heap exploitation. Evaluation Plan: The effectiveness of the platform will be measured based on the number of registered users and their level of engagement and activity.

Positively Impacting Curricula and Beyond. The PIs have created new courses, e.g., *Software Vulnerabilities and Security*, *Advanced Software Security*, *Trusted Computing and Execution*, *IoT Security*, using materials from both the literature and the PIs' research results. The research results from this project and PWNIoT.ACADEMY will be naturally integrated into classroom teaching as demos, homework, projects, and exams. Because systems and software security is difficult to learn, PI Zhao publicizes his course recordings, talks, and training recordings at the YouTube channel [94], which has 400+ subscribers, 30,000+ views, and 3,000+ hours of watch time at the time of submitting this proposal. To reach a wider audience, the PIs will continue to create research, education, and scientific outreach videos on this channel. Evaluation Plan: The courses will be formally assessed by the success of classroom teaching with student evaluations. The number of subscribers and viewership statistics measure the impacts of the YouTube channel.

Economic/Societal Impacts and Technology Transfer. MCU-powered embedded and IoT devices are omnipresent. Meanwhile, the IoT security spending amounts to \$1.2 billion in 2021 [95]. The research activities will fundamentally influence the way MCU security solutions are built and used. As a result, the research not only saves costs in cyberattacks but also has broader societal impacts on energy, transportation, healthcare, public safety, and many other sectors. Our prior work on software-based mitigation for cache side-channel attacks [40] has been evaluated on and integrated into Samsung Tizen OS [57] for wearable

devices, tablets, smart TVs, and in-vehicle infotainment systems. PI Kirda has released and maintained open services (e.g., ANUBIS [96], EXPOSURE [97]) that have been used by hundreds of thousands of users, and that have resulted in industrial products that have protected millions of end-users at Lastline, Inc.

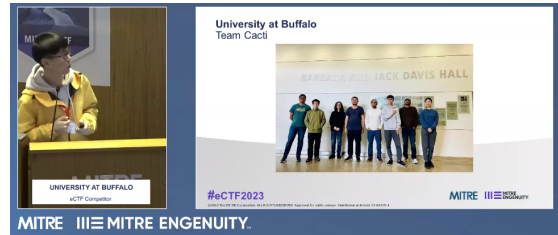
Cybersecurity Training and CTF Teams with Underrepresented, Undergraduate, and High School Students. Lead PI Zhao has founded the *Cacti* hacking team with 300+ registered undergraduate and graduate students and over five high school students (Williamsville East high school, Buffalo; Brighton high school, Rochester; and Agartala high school, India). Team *Cacti* placed 3rd in the MITRE embedded CTF (eCTF) 2019, 6th in 2020, 9th in 2021, 5th in 2022, 4th in 2023 (as shown in Figure 9a and Figure 9c), and 4th in 2024. As shown in Figure 9b, PI Zhao’s high school advisee Puhabi Chakraborti (female) was conferred the Pradhan award by the Prime Minister Modi of India in 2022 [98] and received full scholarship to attend University at Buffalo in 2024. Evaluation Plan: The cybersecurity training impacts will be evaluated by student engagement, the number of CTFs team *Cacti* participates, and the team’s standings in the CTFs. The research experience with underrepresented, undergraduate, and high school students will be evaluated by publications.

Rank	Team	Scoreboard Score	Final Score
1	Carnegie Mellon University	25098	20158
2	University of California Santa Cruz	14476	17167
3	University of Illinois at Urbana-Champaign	9743	12586
4	University at Buffalo	9232	11885
5	Indian Institute of Technology Madras	8933	11346
6	Purdue University	7328	10419
7	Michigan State University	5769	8680
8	Worcester Polytechnic Institute	6221	8576
9	University of Massachusetts Amherst	3972	5899
10	Singapore Management University	3447	5210
11	Tufts University	2647	4676
12	University of Washington	3797	4563
13	Virginia Tech	1590	4052
14	University of New Haven	2447	4044
15	Texas A&M University	2067	3985
16	Florida Atlantic University	2605	3969
17	University of Colorado, Colorado Springs 1	1437	3816
18	University of Colorado, Colorado Springs 2	2369	3743
19	Morgan State University	1496	2620
20	University of California Irvine	1468	1915



(a) 4th in MITRE eCTF’23

(b) Puhabi



(c) Team Cacti in eCTF’23 presentation

Figure 9: CTF Team and High School Advisee

7 Proposed Project Timeline

Figure 10 provides a Gantt chart for the tasks outlined in this proposal. We will spend around two years on each research thrust with a half year overlap. We will develop the CTF platform and challenges in the first two years, after which we will update them based on feedback. We integrate the research results and the CTF platform into course modules in participating institutes during semesters. We will lead the CTF training during semesters. Mentoring and recruiting PhD and minority students will continue in parallel throughout the project.

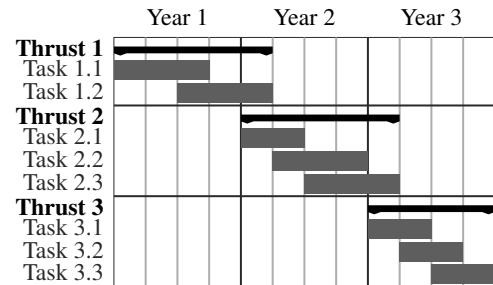


Figure 10: Proposed project timeline

8 Results from Prior NSF Support

PI Zhao: “*CAREER: SaTC: Rethinking Trusted Execution Environments for Embedded and IoT Systems*” (CNS-2237238, \$564,748, 2/2023 - 1/2028). Intellectual Merit: Studying a systematic research approach to increase the trustworthiness and deployability of TEEs for embedded and IoT devices. Broader Impacts: Broader significance, beyond securing the IoT infrastructure, includes training the next generation of cybersecurity researchers, educators, and practitioners with deep theoretical understandings and practical skills. Research Outcome: Return-to-non-secure attacks on ARM Cortex-M TrustZone [43].

Co-PI Kirda: “*SaTC: CORE: Medium: Rethinking Fuzzing for Security*” (CNS-2031390, \$599,766, 8/2020 - 9/2023). Intellectual Merit: Studying new approaches, techniques, and tools that revolutionize fuzzing and make the nearly random testing process more intelligent and targeted. Broader Impacts: This project provides a group of education, training, and research opportunities for both undergraduate and graduate students. Through industry outreach, the investigators pursue technology transfers. Research Outcome: Semi-hosted fuzz testing for embedded applications [47].

References

- [1] P. Sparks, “The route to a trillion devices.” <https://community.arm.com/iot/b/blog/posts/whitepaper-the-route-to-a-trillion-devices>. Online; accessed 20 Apr 2021.
- [2] Arm, “Armv8-M Architecture Technical Overview.” https://community.arm.com/cfs-file/__key/telligent-evolution-components-attachments/01-2142-00-00-00-00-66-90/Whitepaper-_2D00_-Armv8_2D00_M-Architecture-Technical-Overview.pdf.
- [3] E. Systems, “Overview of Xtensa ISA.” https://dl.espressif.com/github_assets/espressif/xtensa-isa-doc/releases/download/latest/Xtensa.pdf.
- [4] Toshiba, “Toshiba Microcontrollers.” <https://toshiba.semicon-storage.com/us/semiconductor/product/microcontrollers.html>.
- [5] Ti, “TI Microcontrollers.” <https://www.ti.com/microcontrollers-mcus-processors/overview.html>.
- [6] Renesas, “Renesas Microcontrollers.” https://www.renesas.com/en/products/microcontrollers-microprocessors?utm_campaign=mcu_leader&utm_source=google_us&utm_medium=search_ad&utm_content=lp&gad_source=1&gclid=Cj0KCQjwrp-3BhDgARIsAEWJ6Sxb9h4ttXvrMKr2DVQkaAmTAqNoxuIJeZBZAa-ukMHjAWapGFa0x9IaAhqBEALw_wcB.
- [7] Infineon, “32-bit AURIX TriCore Microcontroller.” <https://www.infineon.com/cms/en/product/microcontroller/32-bit-tricore-microcontroller/>.
- [8] Arm, “Arm Partner Ecosystem Catalog.” [https://www.arm.com/partners/catalog/results#sort=date\%20descending&f:armip=\[Cortex-M\]](https://www.arm.com/partners/catalog/results#sort=date\%20descending&f:armip=[Cortex-M]).
- [9] Arm, “The Arm ecosystem ships a record 6.7 billion Arm-based chips in a single quarter.” <https://www.arm.com/company/news/2021/02/arm-ecosystem-ships-record-6-billion-arm-based-chips-in-a-single-quarter>.
- [10] Arm, “Arm Partners Have Shipped 200 Billion Chips.” <https://www.arm.com/blogs/blueprint/200bn-arm-chips>.
- [11] FDA, “Certain Medtronic MiniMed Insulin Pumps Have Potential Cybersecurity Risks: FDA Safety Communication.” <https://www.fda.gov/medical-devices/safety-communications/certain-medtronic-minimed-insulin-pumps-have-potential-cybersecurity-risks-fda-safety-communication>, 2019.
- [12] I. Stelliou, P. Kotzanikolaou, M. Psarakis, C. Alcaraz, and J. Lopez, “A survey of iot-enabled cyberattacks: Assessing attack paths to critical infrastructures and services,” *IEEE Communications Surveys & Tutorials*, vol. 20, no. 4, pp. 3453–3495, 2018.
- [13] M. Warner, “S.734 - IoT Cybersecurity Improvement Act of 2019.” <https://www.congress.gov/bill/116th-congress/senate-bill/734>, 2019.

- [14] MITRE, “CVSS Scores Between 2017 and 2023.” https://www.cvedetails.com/cvss-score-charts.php?fromform=1&vendor_id=&product_id=&startdate=2017-01-01&enddate=2023-05-30.
- [15] MITRE, “CVE database.” <https://cve.mitre.org/>.
- [16] A. van de Ven and I. Molnar, “Exec shield.” http://www.redhat.com/f/pdf/rhel/WHP0006US_Execshield.pdf, 2004.
- [17] Intel, “A Technical Look at Intel’s Control-flow Enforcement Technology.” <https://www.intel.com/content/www/us/en/developer/articles/technical/technical-look-control-flow-enforcement-technology.html>. Online; accessed 13 Apr 2023.
- [18] “Rust, A language empowering everyone to build reliable and efficient software.” <https://www.rust-lang.org/>.
- [19] C. Cowan, C. Pu, D. Maier, J. Walpole, P. Bakke, S. Beattie, A. Grier, P. Wagle, Q. Zhang, and H. Hinton, “Stackguard: Automatic adaptive detection and prevention of buffer-overflow attacks,” in *USENIX security symposium*, vol. 98, pp. 63–78, San Antonio, TX, 1998.
- [20] M. Abadi, M. Budiu, Ú. Erlingsson, and J. Ligatti, “Control-flow integrity principles, implementations, and applications,” *ACM Transactions on Information and System Security (TISSEC)*, vol. 13, no. 1, p. 4, 2009.
- [21] G. Tan, “Principles and implementation techniques of software-based fault isolation,” *Foundations and Trends® in Privacy and Security*, vol. 1, no. 3, pp. 137–198, 2017.
- [22] H. Shacham, M. Page, B. Pfaff, E.-J. Goh, N. Modadugu, and D. Boneh, “On the effectiveness of address-space randomization,” in *Proceedings of the 11th ACM conference on Computer and communications security*, pp. 298–307, 2004.
- [23] J. Shi, L. Guan, W. Li, D. Zhang, P. Chen, and N. Zhang, “Harm: Hardware-assisted continuous re-randomization for microcontrollers,” in *2022 IEEE 7th European Symposium on Security and Privacy (EuroS&P)*, pp. 520–536, IEEE, 2022.
- [24] P. Yuan, Q. Zeng, and X. Ding, “Hardware-assisted fine-grained code-reuse attack detection,” in *International Symposium on Recent Advances in Intrusion Detection*, pp. 66–85, Springer, 2015.
- [25] Y. Xia, Y. Liu, H. Chen, and B. Zang, “CFIMon: Detecting violation of control flow integrity using performance counters,” in *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2012)*, pp. 1–12, IEEE, 2012.
- [26] V. Van der Veen, D. Andriesse, E. Göktaş, B. Gras, L. Sambuc, A. Slowinska, H. Bos, and C. Giuffrida, “Practical context-sensitive cfi,” in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pp. 927–940, 2015.
- [27] Y. Liu, P. Shi, X. Wang, H. Chen, B. Zang, and H. Guan, “Transparent and efficient cfi enforcement with intel processor trace,” in *2017 IEEE International Symposium on High performance computer architecture (HPCA)*, pp. 529–540, IEEE, 2017.
- [28] X. Ge, W. Cui, and T. Jaeger, “Griffin: Guarding control flows using intel processor trace,” *ACM SIGPLAN Notices*, vol. 52, no. 4, pp. 585–598, 2017.

- [29] Y. Gu, Q. Zhao, Y. Zhang, and Z. Lin, “PT-CFI: Transparent backward-edge CFVD using intel processor trace,” in *Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy*, pp. 173–184, 2017.
- [30] T. Nyman, J.-E. Ekberg, L. Davi, and N. Asokan, “CFI CaRE: Hardware-supported call and return enforcement for commercial microcontrollers,” in *International Symposium on Research in Attacks, Intrusions, and Defenses (RAID)*, 2017.
- [31] L. Szekeres, M. Payer, T. Wei, and D. Song, “Sok: Eternal war in memory,” in *IEEE Symposium on Security and Privacy*, pp. 48–62, IEEE, 2013.
- [32] M. Gutierrez, Z. Zhao, A. Doupé, Y. Shoshitaishvili, and G.-J. Ahn, “Cachelight: Defeating the cachekit attack,” in *Proceedings of the 2018 Workshop on Attacks and Solutions in Hardware Security*, pp. 65–74, 2018.
- [33] P. Zhang, H. Cho, Z. Zhao, A. Doupé, and G.-J. Ahn, “icore: continuous and proactive extrospection on multi-core iot devices,” in *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, pp. 851–860, 2019.
- [34] Z. Zhao, G.-J. Ahn, and H. Hu, “Automatic extraction of secrets from malware,” in *Working Conference on Reverse Engineering (WCRE)*, pp. 159–168, IEEE, 2011.
- [35] Z. Zhao and G.-J. Ahn, “Using instruction sequence abstraction for shellcode detection and attribution,” in *1st IEEE Conference on Communications and Network Security (CNS)*, 2013.
- [36] J. Tang, X. Cui, Z. Zhao, S. Guo, X. Xu, C. Hu, T. Ban, and B. Mao, “NIVAnalyzer: a tool for automatically detecting and verifying next-intent vulnerabilities in Android apps,” in *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pp. 492–499, IEEE, 2017.
- [37] Y. Jing, Z. Zhao, G.-J. Ahn, and H. Hu, “Morpheus: Automatically generating heuristics to detect android emulators,” in *30th Annual Computer Security Applications Conference (ACSAC)*, 2014.
- [38] Y. Jing, G.-J. Ahn, Z. Zhao, and H. Hu, “Riskmon: Continuous and automated risk assessment for mobile applications,” in *ACM Conference on Data and Application Security and Privacy (CODASPY)*, ACM, 2014.
- [39] Y. Jing, G.-J. Ahn, Z. Zhao, and H. Hu, “Towards automated risk assessment and mitigation of mobile applications,” *IEEE Transactions on Dependable and Secure Computing (TDSC)*, 2015.
- [40] H. Cho, J. Park, D. Kim, Z. Zhao, Y. Shoshitaishvili, A. Doupé, and G.-J. Ahn, “Smokebomb: effective mitigation against cache side-channel attacks on the arm architecture,” in *Proceedings of the International Conference on Mobile Systems, Applications, and Services (MobiSys)*, pp. 107–120, 2020.
- [41] H. Cho, P. Zhang, D. Kim, J. Park, C.-H. Lee, Z. Zhao, A. Doupé, and G.-J. Ahn, “Prime+Count: Novel Cross-world Covert Channels on ARM TrustZone,” in *Proceedings of the Annual Computer Security Applications Conference (ACSAC)*, pp. 441–452, ACM, 2018.
- [42] H. Li, Q. Huang, F. Ding, H. Hu, L. Cheng, G. Gu, and Z. Zhao, “Understanding and detecting remote infection on linux-based iot devices,” in *Proceedings of the ACM on Asia Conference on Computer and Communications Security (AsiaCCS)*, pp. 873–887, 2022.
- [43] Z. Ma, X. Tan, L. Ziarek, N. Zhang, H. Hu, and Z. Zhao, “Return-to-non-secure vulnerabilities on arm cortex-m trustzone: Attack and defense,” in *ACM/IEEE Design Automation Conference*, 2023.

- [44] Z. Zhao, M. Armanuzzaman, X. Tan, and Z. Ma, “Trusted Execution Environments in Embedded and IoT Systems: A CactiLab Perspective,” in *IEEE International Symposium on Secure and Private Execution Environment Design*, 2024.
- [45] Y. Wang, C. Lemieux Mack, X. Tan, N. Zhang, Z. Zhao, S. Baruah, and B. C. Ward, “InsectACIDE: Debugger-Based Holistic Asynchronous CFI for Embedded System,” in *IEEE Real-Time and Embedded Technology and Applications Symposium*, 2024.
- [46] M. Armanuzzaman, A.-R. Sadeghi, and Z. Zhao, “Building Your Own Trusted Execution Environments Using FPGA,” in *ACM ASIA Conference on Computer and Communications Security*, 2024.
- [47] A. Mera, C. Liu, R. Sun, E. Kirda, and L. Lu, “SHiFT: Semi-hosted Fuzz Testing for Embedded Applications,” in *USENIX Security Symposium*, pp. 5323–5340, 2024.
- [48] A. Mera, Y. H. Chen, R. Sun, E. Kirda, and L. Lu, “D-box: Dma-enabled compartmentalization for embedded applications,” *arXiv preprint arXiv:2201.05199*, 2022.
- [49] X. Tan, Z. Ma, S. Pinto, L. Guan, N. Zhang, J. Xu, Z. Lin, H. Hu, and Z. Zhao, “SoK: Where’s the “up”?! A Comprehensive (bottom-up) Study on the Security of Arm Cortex-M Systems,” in *USENIX WOOT Conference on Offensive Technologies*, 2024.
- [50] N. Nino, R. Lu, W. Zhou, K. H. Lee, Z. Zhao, and L. Guan, “Unveiling IoT Security in Reality: A Firmware-Centric Journey,” in *USENIX Security Symposium*, 2024.
- [51] P. Zhang, B. Ngabonziza, H. Cho, Z. Zhao, A. Doupé, and G.-J. Ahn, “SeCore: Continuous Extrospection with High Visibility on Multi-core ARM Platforms,” in *Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy (CODASPY)*, pp. 161–163, ACM, 2018.
- [52] H. Feng, H. Li, X. Pan, and Z. Zhao, “A formal analysis of the fido uaf protocol,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2021.
- [53] J. Lamp, C. E. Rubio-Medrano, , Z. Zhao, and G.-J. Ahn, “Exsol: Collaboratively assessing cybersecurity risks for protecting energy delivery systems,” in *Workshop on Modeling and Simulation of Cyber-Physical Energy Systems*, April 2019.
- [54] G.-J. Ahn and Z. Zhao, “Methods, systems, and media for measuring quality of gesture-based passwords,” June 30 2015. US Patent US9069948 B2.
- [55] H. Tu, A. Doupe, G.-J. Ahn, and Z. Zhao, “Systems and methods for authenticating caller identity and call request header information for outbound telephony communications,” Sept. 14 2017. US Patent App. 15/459,597.
- [56] iSign, “iSign international.” <https://www.isignintl.com/>, 2017.
- [57] Samsung, “Tizen - an open source, standards-based software platform.” <https://www.tizen.org/>.
- [58] “Armv8.1-M Pointer Authentication and Branch Target Identification Extension.” <https://community.arm.com/developer/ip-products/processors/b/processors-ip-blog/posts/armv8-1-m-pointer-authentication-and-branch-target-identification-extension>.

- [59] A. Machiry, E. Gustafson, C. Spensky, C. Salls, N. Stephens, R. Wang, A. Bianchi, Y. R. Choe, C. Kruegel, and G. Vigna, “BOOMERANG: Exploiting the Semantic Gap in Trusted Execution Environments,” in *Proceedings of the 2017 Annual Network and Distributed System Security Symposium (NDSS)*, (San Diego, CA), Feb.–Mar. 2017.
- [60] V. P. Kemerlis, G. Portokalidis, and A. D. Keromytis, “kGuard: Lightweight Kernel Protection against Return-to-User Attacks,” in *USENIX Security Symposium*, pp. 459–474, 2012.
- [61] Arm, “Arm Cortex-M55 Processor Technical Reference Manual Revision r0p2.” <https://developer.arm.com/documentation/101051/0002/>.
- [62] Arm, “Arm Cortex-M55 Processor Technical Reference Manual Revision r0p2.” <https://developer.arm.com/documentation/101924/0002/?lang=en>.
- [63] Arm, “AN505: Cortex™-M33 with IoT kit FPGA for MPS2+ Version 2.0,” 2017. <https://developer.arm.com/tools-and-software/development-boards/fpga-prototyping-boards/download-fpga-images>.
- [64] T. H. Dang, P. Maniatis, and D. Wagner, “The performance cost of shadow stacks and stack canaries,” in *Proceedings of the 10th ACM Symposium on Information, Computer and Communications Security*, pp. 555–566, 2015.
- [65] N. Burow, X. Zhang, and M. Payer, “Sok: Shining light on shadow stacks,” in *2019 IEEE Symposium on Security and Privacy (SP)*, pp. 985–999, IEEE, 2019.
- [66] N. S. Almakhdhub, A. A. Clements, S. Bagchi, and M. Payer, “ μ RAI: Securing Embedded Systems with Return Address Integrity,” in *Network and Distributed Systems Security (NDSS) Symposium*, 2020.
- [67] R. J. Walls, N. F. Brown, T. Le Baron, C. A. Shue, H. Okhravi, and B. C. Ward, “Control-flow integrity for real-time embedded systems,” in *31st Euromicro Conference on Real-Time Systems (ECRTS 2019)*, Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2019.
- [68] J. Zhou, Y. Du, Z. Shen, L. Ma, J. Criswell, and R. J. Walls, “Silhouette: Efficient protected shadow stacks for embedded systems,” in *USENIX Security Symposium*, pp. 1219–1236, 2020.
- [69] Y. Du, Z. Shen, K. Dharsee, J. Zhou, R. J. Walls, and J. Criswell, “Holistic Control-Flow Protection on Real-Time Embedded Systems with Kage,” in *31st USENIX Security Symposium (USENIX Security 22)*, USENIX Association, 2022.
- [70] Intel, “Intel 64 and IA-32 Architectures Software Developer’s Manual Volume 3C: System Programming Guide, Part 3.” <https://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-software-developer-vol-3c-part-3-manual.pdf>.
- [71] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, and G. Vigna, “SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis,” in *IEEE Symposium on Security and Privacy*, 2016.
- [72] C. Zhang, T. Wei, Z. Chen, L. Duan, L. Szekeres, S. McCamant, D. Song, and W. Zou, “Practical control flow integrity and randomization for binary executables,” in *2013 IEEE Symposium on Security and Privacy*, pp. 559–573, IEEE, 2013.

- [73] M. Zhang, R. Qiao, N. Hasabnis, and R. Sekar, "A platform for secure static binary instrumentation," in *ACM SIGPLAN/SIGOPS international conference on Virtual execution environments*, 2014.
- [74] R. Wang, Y. Shoshitaishvili, A. Bianchi, A. Machiry, J. Grosen, P. Grosen, C. Kruegel, and G. Vigna, "Ramblr: Making Reassembly Great Again," in *Network and Distributed Systems Security (NDSS) Symposium*, 2017.
- [75] L. Zhu, X. Fu, Y. Yao, Y. Zhang, and H. Wang, "FloT: Detecting the memory corruption in lightweight IoT device firmware," in *IEEE International Conference On Trust, Security And Privacy (TrustCom)*, 2019.
- [76] T. Frassetto, P. Jauernig, D. Koisser, and A.-R. Sadeghi, "CfInsight: A comprehensive metric for CFI policies," in *Network and Distributed System Security Symposium (NDSS)*, 2022.
- [77] T. Kawada, S. Honda, Y. Matsubara, and H. Takada, "Tzmcfi: Rtos-aware control-flow integrity using trustzone for armv8-m," *International Journal of Parallel Programming*, pp. 1–21, 2020.
- [78] T. Abera, N. Asokan, L. Davi, J.-E. Ekberg, T. Nyman, A. Paverd, A.-R. Sadeghi, and G. Tsudik, "C-FLAT: control-flow attestation for embedded systems software," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 743–754, 2016.
- [79] Z. Sun, B. Feng, L. Lu, and S. Jha, "Oat: Attesting operation integrity of embedded devices," in *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 1433–1449, IEEE, 2020.
- [80] Github, "Open Syringe Pump." <https://github.com/manimino/OpenSyringePump>.
- [81] G. Dessouky, S. Zeitouni, T. Nyman, A. Paverd, L. Davi, P. Koeberl, N. Asokan, and A.-R. Sadeghi, "Lo-fat: Low-overhead control flow attestation in hardware," in *Proceedings of the 54th Annual Design Automation Conference 2017*, pp. 1–6, 2017.
- [82] N. Yadav and V. Ganapathy, "Whole-program control-flow path attestation," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023.
- [83] "Embench: A Modern Embedded Benchmark Suite." <https://www.embench.org/>.
- [84] CactiLab, "Secure Common Embedded Wireless Link - Team Cacti." <https://github.com/CactiLab/2021-ectf-UB-Cacti-design>, 2021.
- [85] CactiLab, "Secure Avionics Flight Firmware Installation Routine - Team Cacti." <https://github.com/CactiLab/2022-ectf-UB-Cacti-design>, 2022.
- [86] Ghidra. <https://ghidra-sre.org/>.
- [87] Angr. <https://angr.io/>.
- [88] FreeRTOS, "Freertos." <https://www.freertos.org/>. Online; accessed 19 Jan 2021.
- [89] Arm, "Mbed OS." <https://os.mbed.com/mbed-os/>.
- [90] N. S. Almahdhub, A. A. Clements, M. Payer, and S. Bagchi, "Benchiot: A security benchmark for the internet of things," in *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp. 234–246, IEEE, 2019. Source code available: <https://github.com/embedded-sec/BenchIoT>.

- [91] J. Pallister, S. Hollis, and J. Bennett, “BEEBS: open benchmarks for energy measurements on embedded platforms,” *arXiv preprint arXiv:1308.5174*, 2013. Source code available: <https://github.com/mageec/beebs>.
- [92] Y. Shoshitaishvili and C. Nelson, “pwn.college.” <https://pwn.college/>.
- [93] “Overthewire: Wargames.” <https://overthewire.org/>.
- [94] Z. Zhao, “My lecture recordings on youtube.” <https://www.youtube.com/channel/UCkSeVUu-AxytXqalx66j7Eg>, 2016.
- [95] Statista, “Internet of Things security spending worldwide from 2016 to 2021.” <https://www.statista.com/statistics/543089/iot-security-spending-worldwide/>.
- [96] M. Egele, T. Scholte, E. Kirda, and C. Kruegel, “A survey on automated dynamic malware-analysis techniques and tools,” *ACM Comput. Surv.*, vol. 44, no. 2, pp. 6:1–6:42, 2012.
- [97] L. Bilge, E. Kirda, C. Kruegel, and M. Balduzzi, “EXPOSURE: finding malicious domains using passive DNS analysis,” in *Proceedings of the Network and Distributed System Security Symposium, NDSS, San Diego, California, USA, 6th February - 9th February*, The Internet Society, 2011.
- [98] E. Rise, “Proud moment for tripura, puhabi chakraborti wins rashtriya baal puraskar.” <https://theriseeast.com/tripura/proud-moment-for-tripura-puhabi-chakraborti-wins-rashtriya-baal-puraskar/>. Online; accessed Mar 2022.